



Smart Grid Energy Monitoring & Billing: A Lambda-Architecture Data Pipeline

Mini Project Assessment

EC8203 Applied Big Data Engineering

A mini project report submitted to the

Department of Electrical and Information Engineering
Faculty of Engineering
University of Ruhuna
Sri Lanka

by

Sewvandi M.A.K.	- EG/2021/4808
Peiris P.R.S.	- EG/2021/4706

September 2026

Contents

1. Introduction and Use Case.....	3
1.1 Business question and derived requirements.....	3
1.2 Data sources (simulated in Python).....	3
1.3 Simulated clock and assumptions.....	4
2. Architecture Decision: Lambda vs Kappa.....	4
2.1 Why Lambda fits this problem.....	4
2.2 The rejected alternative: Kappa.....	5
2.3 Trade-offs we accepted and how we contain them.....	5
3. System Architecture	6
4. Technology Stack and Justification.....	6
5. Implementation.....	7
5.1 Ingestion.....	7
5.2 Speed layer	7
5.3 Batch layer and orchestration	8
5.4 Storage and serving layer	9
6. Observability Design.....	9
6.1 Failure drill: stopping the streaming source	10
7. Testing and Reproducibility	11
8. Results	12
8.1 Live dashboard and real-time API.....	12
8.2 Daily billing and solar-contribution report (batch view).....	13
8.3 Lambda merge: provisional bills during the day.....	13
8.4 Speed vs batch reconciliation and data quality.....	14
8.5 Alerts, orchestration and monitoring.....	15
9. Limitations, Trade-offs and Production-Scale Changes.....	15
10. Conclusion.....	16
Individual contributions	16

1. Introduction and Use Case

A utility company wants to see, in real time, how much electricity each part of its network is using and how much of it is coming from household solar panels. Once a day it also has to apply the latest tariff and billing data to that consumption and produce a bill for every household. We implemented **Use Case 3 (Smart Grid Energy Monitoring and Billing)** with both of the required kinds of source: a continuous stream of smart-meter readings and a tariff file that arrives once per day.

1.1 Business question and derived requirements

Business question: “What is the current grid load and renewable contribution by zone, and what will each household's bill look like once daily tariff data is applied to their consumption?” We broke this question down into functional (FR) and non-functional (NFR) requirements. Every later design decision refers back to this table:

ID	Requirement	Where it is met
FR1	Live load, solar generation and renewable share for each grid zone	Speed layer, shown on the live dashboard and through the live grid API
FR2	Alert when the renewable contribution is considerably low	Low-renewable alert rule (checked in daylight only)
FR3	Daily bill and solar-contribution report per household once the tariff arrives	Batch layer, stored in the billing tables and published as CSV and HTML report files
FR4	An estimate of today's bill before the day has finished	Lambda merge in the serving layer (provisional bill)
FR5	Cope with dirty data in both feeds	Quarantine of bad readings, duplicate removal, tariff validation with fallback
NFR1 Latency	Live view within seconds; bills within minutes of the tariff file	10-second micro-batches; the batch pipeline checks for work every minute
NFR2 Correctness / replay	Bills must be exact, reproducible and possible to re-run	Unchangeable Parquet data lake; each day is rewritten in full when re-run
NFR3 Consistency	Readers never see half a day; speed and batch results can be compared	Each day is written in one database transaction; a reconciliation record per day
NFR4 Observability	Detect and diagnose a failure in any stage	Structured logs, heartbeats, seven alert rules and Prometheus
NFR5 Cost / operability	Runs on one laptop with one command	Docker Compose with nine containers

1.2 Data sources (simulated in Python)

Streaming source. Twenty-four households spread over four zones (North, South, East and West) each send a reading every two seconds. A reading contains a unique event ID, the meter and household it came from, the energy consumed and the solar energy generated in kWh, the zone, and a timestamp. Consumption follows a morning and an evening peak with random noise and occasional demand spikes. Solar generation follows a daylight curve that depends on how many panels each zone has (South has the fewest) and on a daily cloud level, and every simulated day one zone goes through a heavily overcast spell. About 1% of readings are made faulty on purpose: a missing field, a negative or impossible value, an unreadable timestamp, or an exact duplicate that imitates a message being delivered twice.

Daily-batch source. Twenty seconds after each simulated day ends, this source publishes that day's tariff extract as a CSV file. For every household it lists the tariff rate, the billing tier, whether the household receives a subsidy, the day, and when the file was published. The file is first written under a temporary name and then renamed in one step, and only after that is a small completion marker written, so the batch layer can never pick up a half-written file. A household's tier and subsidy never change, and the daily rate variation is generated from the

household and the day, so producing the same day again gives an identical file. About one file in three contains an invalid rate and some contain a household twice. If the source restarts, it fills in any days it missed.

1.3 Simulated clock and assumptions

- **One simulated day lasts 300 seconds (5 minutes) of real time.** This can be changed in the configuration. Halfway through a simulated day is noon, and for alerting we treat the middle 20% of the day (from 40% to 60% of the way through) as daylight.
- All components count days from one shared start time, which is saved in PostgreSQL when the database is first created. A reading's day number is the number of whole 300-second periods between that start time and the time the reading was taken. Because the reading's own timestamp is used, not the time it happened to be processed, a late reading is still counted in the correct day, and every container agrees on where one day ends and the next begins.
- Billing: $\text{bill} = \max(\text{consumption} - \text{solar}, 0) \times \text{tariff rate} \times (1 - 15\% \text{ if the household is subsidised})$. Surplus solar is not credited. Renewable contribution = $\text{solar} \div \text{consumption}$, capped at 100%. Amounts are in Sri Lankan rupees (LKR). All data is synthetic.

2. Architecture Decision: Lambda vs Kappa

Both architectures start from the same unchangeable log of events. **Kappa** has a single stream-processing path, and when history has to be recomputed it replays the log through that same path. **Lambda** keeps a speed layer that gives fast but approximate results and a batch layer that periodically recomputes exact results from the full stored data, and a serving layer combines the two. We compared them on the four points the marking rubric asks about:

Dimension	What this use case needs	Lambda	Kappa
Latency	Seconds for the grid view; minutes for bills	Speed layer: about 10–15 s end to end (measured). Batch: a bill about 1 minute after the tariff file	Same stream latency; bills also produced by the stream
Replay / correction	Re-bill one day after a corrected tariff file or a code fix	Re-run one Spark job over that one day of stored data; the result simply replaces the old one	Replay the Kafka log from before that day through a new version of the job, then switch outputs
Consistency	Exact, auditable daily bills that appear all at once; duplicates removed	Duplicates removed across the whole day, and the day is written in one transaction	Needs exactly-once state and careful watermarks to close a day
Cost / complexity	Laptop-scale, two-week build, every line explainable	Two processing paths (mitigated by one engine and shared rule modules)	One path, but a long-running stateful join between a stream and a daily file

2.1 Why Lambda fits this problem

- **The two sources are different by nature.** Meter readings are a genuine stream, while the tariff feed is a complete snapshot delivered once a day. Lambda lets us treat each one in its natural form. In Kappa the daily file would have to be pushed into Kafka as a tiny 24-message stream and joined with the meter stream, which means keeping a whole day of state and waiting for a file that only arrives after the day has ended.
- **Billing must be correct and repeatable.** A bill has to be reproducible from the source data. Our batch layer recalculates each household's total from the stored raw readings, removes duplicates across the whole day, and replaces that day's results in one transaction, so running it again for the same day gives exactly the same bills. The speed layer is openly allowed to be approximate: it only removes duplicates inside each small batch and keeps running totals, which can count a reading twice if Kafka delivers it again after a crash.

- **We can measure the difference instead of just claiming it.** For every billed day the batch job records how far the speed layer's total was from the exact batch total. Across the 5 simulated days of our run this difference was between 0.00% and 0.27%, and the batch layer removed 15 duplicate readings that the speed layer had counted (section 8).
- **Reprocessing is cheap and targeted.** Correcting one day means re-reading only that day's stored data, not the whole log. The stored data also outlives Kafka's seven-day retention; a Kappa design would need to keep the Kafka log forever (or add tiered storage) to reprocess older days.
- **The daily work fits an orchestrator naturally.** The batch side is a chain of dependent steps (wait for the file, run Spark, publish the report, re-check the alerts), which benefits directly from Airflow's retries, backfill and web interface.

2.2 The rejected alternative: Kappa

Kappa would be the better choice if **all** inputs were naturally streams, if bills were needed continuously rather than once a day, or if keeping two processing paths in line were the biggest cost. Its real strengths are a single code path, no possibility of the speed and batch views disagreeing, and simpler operation. We rejected it here for three reasons: (1) the daily tariff snapshot is not a natural stream; (2) closing a day exactly inside a stream requires exactly-once state and a watermark tuned to a file that arrives after the day ends; and (3) correcting a single day would mean replaying the log through a new version of the job and swapping the outputs, which is out of proportion for a one-day fix. We still keep the replayable Kafka log, so moving towards Kappa later remains possible.

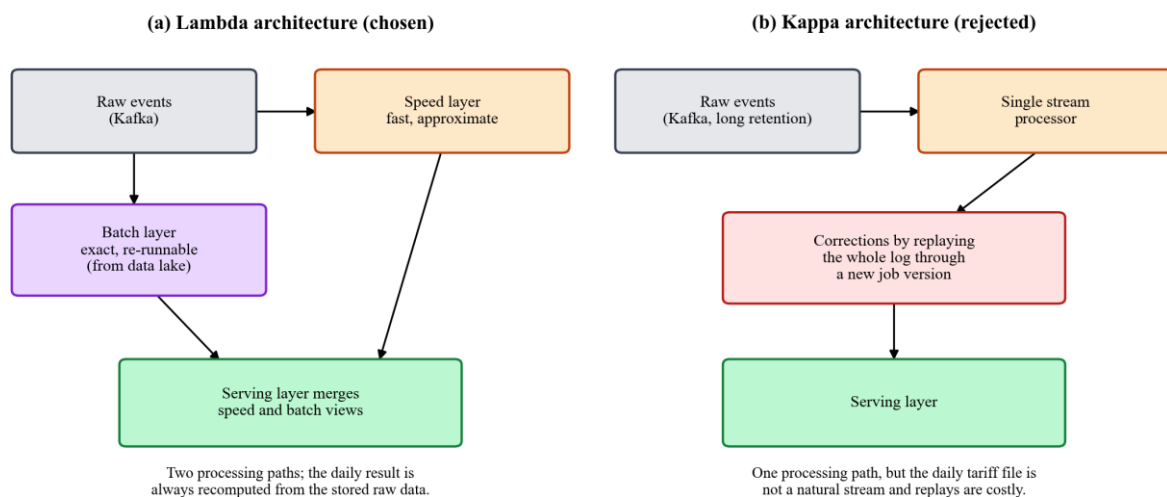


Figure 1. Lambda (chosen) versus Kappa (rejected) for this use case.

2.3 Trade-offs we accepted and how we contain them

- **Two processing paths could drift apart.** We reduce this risk by using Spark for both layers and by sharing the rules between them: the same data-validation rules and the same billing formula are used by the batch job and by the API's provisional bill. The end-to-end test also recalculates every bill with the billing formula and checks that it matches.
- **The live view can be briefly wrong** (duplicates, or readings arriving more than one minute late). We accept this because the live view is only used for monitoring and for estimates that are clearly labelled provisional; final bills come only from the batch layer.
- **Bills are only ready after the day closes (plus about one minute).** This is acceptable because the business asks for daily bills, and the provisional bill covers the need for an estimate during the day.

3. System Architecture

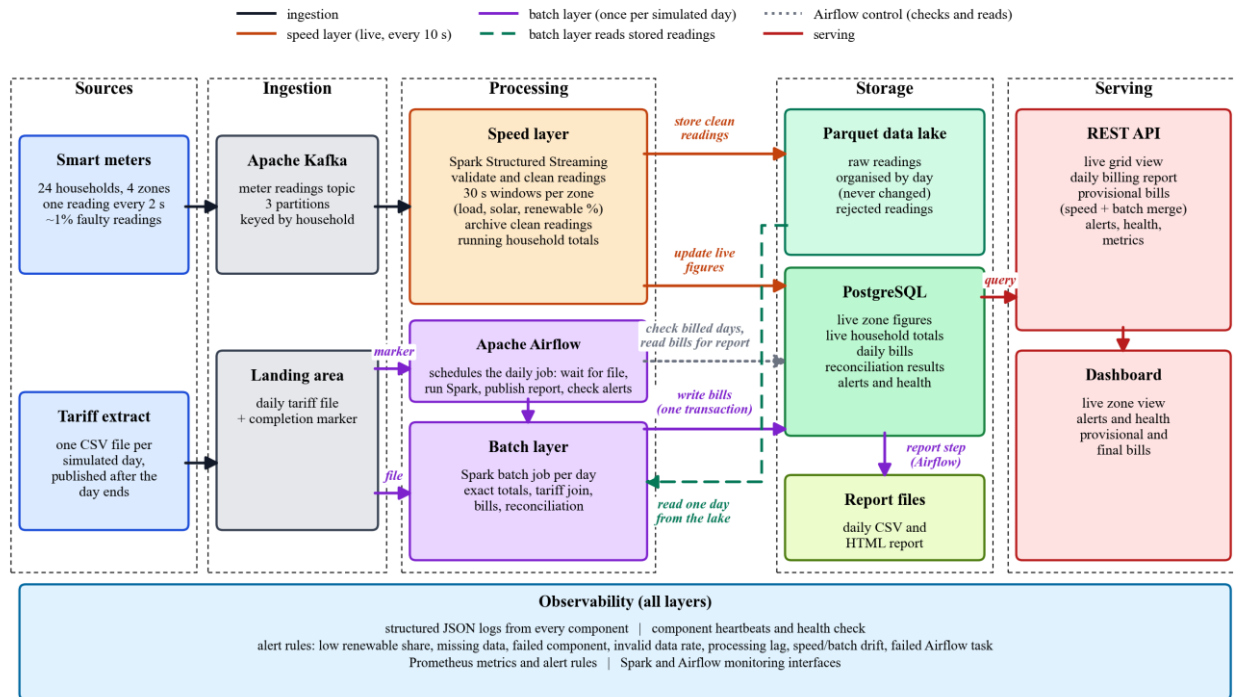


Figure 2. End-to-end architecture across the ingestion, processing, storage and serving layers, with observability as a cross-cutting concern.

How data flows through the system. (1) The meter simulator sends every reading to a Kafka topic with three partitions, using the household as the message key so that each household's readings stay in order. (2) The speed layer runs two Spark streaming queries that always continue from their last saved position in Kafka. The first calculates rolling figures for each zone and keeps them up to date in PostgreSQL. The second puts faulty readings aside, adds clean readings to the Parquet data lake (organised by day), keeps a running total for each household, and records data-quality and delay figures. (3) After a day ends, the tariff source drops that day's CSV file and its completion marker. (4) The Airflow pipeline checks PostgreSQL for the oldest finished day that has not been billed yet, waits for that day's marker and starts the Spark batch job. (5) The batch job reads that day from the data lake and writes the whole day to PostgreSQL in a single transaction; Airflow then builds that day's CSV and HTML report files from the stored bills. (6) The FastAPI service reads PostgreSQL and serves the live view, the daily view and the combination of the two; the dashboard and Prometheus read from the API.

4. Technology Stack and Justification

Layer	Choice	Why this fits the use case	Alternatives considered
Ingestion	Apache Kafka 3.7 (one broker, without ZooKeeper), three partitions, household as the message key, every message confirmed by the broker, seven-day retention	A durable buffer that can be replayed for continuous telemetry. Keying by household keeps each household's readings in order while spreading the 24 meters across partitions so they can be read in parallel. Retention lets the speed layer catch up after an outage, because it always resumes from its last saved position	RabbitMQ (no replay or stored positions); Kafka with ZooKeeper (one more component to run)
Stream processing	Spark Structured Streaming 3.5	Time windows based on when the reading was taken, with watermarks for late meter data; saved positions so nothing is read twice or skipped; each micro-batch can be written to several places at once; and the same programming interface as the batch job, so one engine to learn and defend	Apache Storm (processes one message at a time and has no matching batch interface); Flink (not in the preferred stack, and its lower latency is not needed here)

Layer	Choice	Why this fits the use case	Alternatives considered
Batch processing	Spark, submitted as a job running on two local cores	Reads one day of Parquet data, removes duplicates across the day and joins it with the tariff file; the same code would run on a cluster unchanged	Plain pandas (would not scale, and would be a different interface from the speed layer)
Orchestration	Apache Airflow 2.9	A daily chain of dependent steps with a file sensor, retries, failure notifications, backfill (it looks back three days) and a web interface for the demo and for diagnosis	Cron (no dependencies, retries or visibility)
Master dataset	Parquet data lake organised by day	Unchangeable, column-based and cheap; recomputing a day reads only that day's files	Keeping raw readings in PostgreSQL (a row store that would grow without limit)
Serving store	PostgreSQL 16	The outputs are small and relational and are read with filters, joins (the provisional bill is live totals multiplied by the latest tariff) and aggregates; transactions, and rows that are updated in place rather than duplicated, mean both layers can safely write the same results again	Cassandra (built for writes, with no joins or transactions, and our reads need both); HDFS or S3 on their own (not a fast query store)
Serving API / UI	FastAPI with a static HTML dashboard	Typed endpoints with automatically generated documentation; the dashboard uses only the public API	Grafana on its own (cannot express the billing merge)
Observability	Structured JSON logs, heartbeats stored in PostgreSQL, a Python alert engine, Prometheus	Rules that understand both the business and the pipeline (a renewable rule that only runs in daylight, a separate silence limit for each component) plus a standard metrics and alerting tool	ELK or Grafana stack (heavier than needed on one machine)
Packaging	Docker Compose (nine services)	One command gives a reproducible environment; images are built from official Apache Kafka, Apache Airflow and Python bases	Manual installation

5. Implementation

5.1 Ingestion

- The meter simulator creates the Kafka topic itself with three partitions and keeps retrying until the broker is ready. Every message must be confirmed by the broker, only one unconfirmed batch is allowed at a time (so a retry can never change the order of a household's readings), and every confirmed or failed send is counted. When the container is stopped it sends whatever is still pending and closes cleanly, and it publishes its delivery counters for Prometheus.
- The tariff source never exposes a partial file (temporary name, one-step rename, then the completion marker last), always produces the same file for the same day, and fills in any days it missed while it was down.

5.2 Speed layer

- On its very first start the speed layer reads Kafka from the beginning, and after that it continues from its saved position (stored next to the data lake). Nothing sent before Spark was ready is lost, and a restart picks up exactly where it stopped.
- **Cleaning:** every reading is checked against a fixed structure and against the shared data-quality rules (missing field, unreadable timestamp, unknown zone, negative value, or more than 50 kWh). Invalid readings are stored in a separate quarantine area together with the reason they failed. Nothing is dropped silently.
- **Windowing and aggregation:** clean readings are grouped per zone into 30-second windows that move forward every 15 seconds, allowing readings to arrive up to one minute late. For each window we

calculate the load, the solar generation, the renewable share and the number of households, and update that zone's row in PostgreSQL (writing the same window twice gives the same result).

- **Master dataset and enrichment:** in the second query every micro-batch has its duplicates removed, each reading is labelled with the simulated day it was taken on, and it is added to that day's part of the raw data lake. The same query adds each household's usage to its running total for the day, records the number of readings, the invalid readings by reason, the duplicates and the delay between a reading being taken and being stored, and sends a heartbeat.

5.3 Batch layer and orchestration

- The batch job reads only the day being billed, removes duplicate readings **across the whole day**, and calculates exact figures for each household: consumption, solar, net consumption, surplus solar, solar-contribution percentage, peak reading and number of readings.
- It reads the tariff file with every column treated as text first, then rejects rows whose rate is blank, not a number, zero or negative, or whose tier is unknown. It keeps one row per household and, when a household's rate is missing or invalid, **falls back to that household's last valid tariff** from an earlier day, marking the bill as using a fallback rate. Households with no usable tariff at all are logged and counted rather than billed with a guess.
- It then joins consumption with tariffs, applies the billing formula, and saves the daily consumption, the validated tariffs, the bills and the reconciliation figures. For that day it deletes any old rows and inserts the new ones **in one transaction**, so the day appears all at once and a re-run simply replaces it. Every run has its own run ID, which appears in every log line and is stored with each bill.
- The Airflow pipeline runs every minute and has six steps: find the oldest finished day that has not been billed and has data (skipping the rest of the run if there is none, and looking back up to three days); wait for that day's tariff file; run the Spark batch job; publish the CSV and HTML report; re-check the alert rules; and log that the run finished. Failed steps are retried twice, 20 seconds apart, and a final failure raises a critical alert. We poll every minute instead of using a daily schedule because, with the compressed clock, a day ends every five minutes at no fixed time of day.

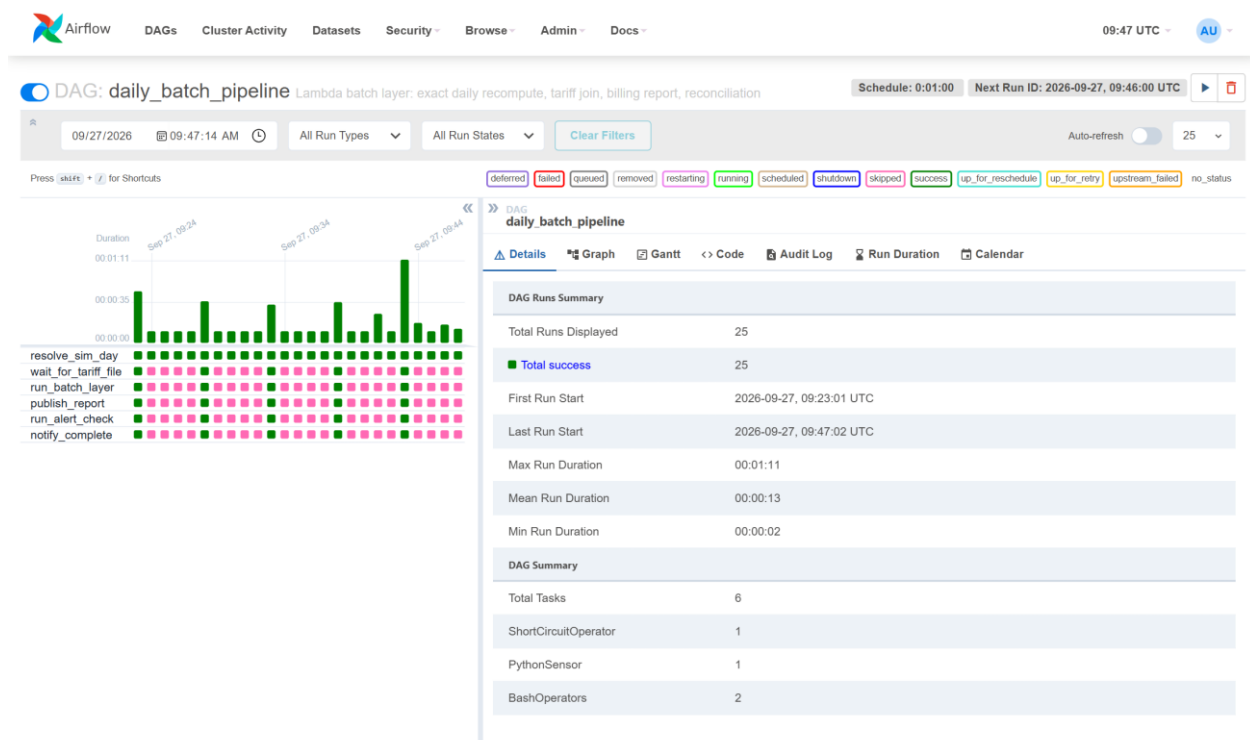


Figure 3. Airflow grid view of the daily batch pipeline: complete runs (green) bill a finished simulated day; checks in between find nothing to do and skip the remaining steps (pink).

5.4 Storage and serving layer

Stored data	Written by	Purpose
Raw readings in Parquet, organised by day	Speed layer	Unchangeable master dataset used by the batch layer
Quarantined readings in Parquet, organised by failure reason	Speed layer	Rejected readings kept for diagnosis
Live zone figures	Speed layer	Load, solar and renewable share per zone and window (kept for 6 hours)
Live household totals	Speed layer	Approximate running totals for the current day
Stream quality figures	Speed layer	Per micro-batch: readings, invalid and duplicate counts, delay
Daily consumption, validated tariffs and bills	Batch layer	The exact daily view and the final bills
Daily reconciliation	Batch layer	Difference between speed and batch totals, plus data-quality counts, per day
Alerts, component heartbeats and the shared clock	Observability / shared	Alert history, component health and the common start time

Endpoint	View	Answers
GET /api/grid/live, /api/grid/history	Speed	Current load, solar and renewable share per zone; the last 15 minutes
GET /api/households/live	Speed	Today's running consumption per household
GET /api/billing/daily, /summary, /days	Batch	Bill and solar contribution per household; totals per zone; list of billed days
GET /api/billing/projection	Merge	Provisional bill so far today = live consumption × latest validated tariff, shown beside the last final bill
GET /api/reconciliation	Batch	Speed-versus-batch difference and data-quality counts
GET /api/alerts, /api/pipeline/status, /health, /metrics	Observability	Open and resolved alerts, heartbeats, stream quality and Prometheus metrics

The live zone query deliberately returns the latest **complete** window for each zone, because a window that is still open holds only a few seconds of data and would under-report the load. If the database is unavailable, the API logs the problem and returns a “service unavailable” response instead of crashing. The same service also hosts the dashboard and the interactive API documentation.

6. Observability Design

Our aim is that any failure, whether a stopped producer, a stuck Spark query, a crashed batch job or a bad input file, is **detected** automatically and can be **diagnosed** from one place. We collect four kinds of signal and use shared identifiers to trace work from one end of the pipeline to the other:

Signal	How it is produced	Why
Structured logs	Every component writes one JSON record per event (time, level, component, message and extra fields) to its console and to its own log file. Each record carries the simulated day and, where relevant, the micro-batch, batch run, reading and household it relates to	Follow one day or one batch run through the producer, Spark, Airflow and the API; easy for a log collector to parse
Heartbeats	Each long-running component regularly records its status and a short detail in PostgreSQL; each component has its own limit for how long it may stay silent	Shows which stage is alive; the batch layer is normally quiet for a whole day, so a single limit for everything would cause false alarms
Pipeline metrics	Stream quality figures for every micro-batch and reconciliation figures for every day, published on the API's metrics endpoint (refreshed from PostgreSQL each	Volume, invalid rate by reason, duplicates, delay, speed-versus-batch difference, delivery failures and API response times

Signal	How it is produced	Why
	time Prometheus reads it) and on the producer's own metrics endpoint	
Alerts	A Python alert engine checks every 15 seconds and after each batch run; Prometheus evaluates matching alert rules	Turns the signals into alerts that are never duplicated, close themselves when the problem clears, and can be sent to a chat webhook

Rule	Condition (default threshold)	Detects
Low renewable share (business rule)	A zone's renewable share is below 10% in the latest complete window, during simulated daylight only	The grid relying on non-renewable supply, e.g. an overcast zone
No data	A component has not sent a heartbeat within its limit (2 minutes for streaming components; 2 simulated days plus 2 minutes for the batch layer)	A stopped or stuck producer, Spark query, alert engine or batch layer
Component failed	A component reports a status other than OK (e.g. the batch job threw an error)	Crashes in components that are still sending heartbeats
High invalid rate	More than 5% of readings quarantined in the last 5 minutes	A problem with the sensors or the data format upstream
High stream lag	Average delay from a reading being taken to being stored is over 60 seconds	The speed layer falling behind
Reconciliation drift	Speed and batch totals for the last billed day differ by more than 10%	The speed layer double counting or losing data
Pipeline task failed	An Airflow step failed after all its retries	A failure in the batch orchestration

The database only allows one open alert of each type for the same zone or component. While the condition continues, that alert's latest time and value are updated, and when the condition clears the alert is closed automatically. Every change is logged, and it is also posted to a Slack- or Teams-compatible webhook if one is configured.

Tracing. Every reading has its own event ID, every speed-layer micro-batch a batch number and every batch-layer run a run ID. These, together with the simulated day and the household, are written into the structured logs and stored with the results (each bill records the run that produced it). This lets us follow a single reading, micro-batch or daily billing run from the producer through Kafka, Spark and Airflow to PostgreSQL and the API, which is how a failure is pinned down to the stage where it happened.

6.1 Failure drill: stopping the streaming source

To show that failures are detected and can be diagnosed, we stopped the meter producer container at 09:26:21 UTC. Within about two minutes the alert engine raised two “no data” alerts, one for the producer and one for the speed layer (which had nothing left to process), and the health check marked both as stale. After we restarted the producer, the speed layer continued from its saved position in Kafka and both alerts closed on their own, with no manual action.

Time (UTC)	Observation
09:26:21	Meter producer container stopped (the streaming source goes silent)
09:28:23	Health check marks the meter producer as stale (last heartbeat 127 s ago); overall status becomes DEGRADED
09:28:33	Health check marks the speed layer as stale (last heartbeat 122 s ago)
09:28:33	Critical “no data” alert raised for the meter producer (silent for 2.2 minutes against a 2-minute limit)
09:28:43	Critical “no data” alert raised for the speed layer (silent for 2.0 minutes)
09:28:51	Dashboard screenshot taken: status DEGRADED, both alerts listed (Figure 4)
09:29:52	Meter producer container started again
09:30:12	Both alerts closed automatically; health check back to OK

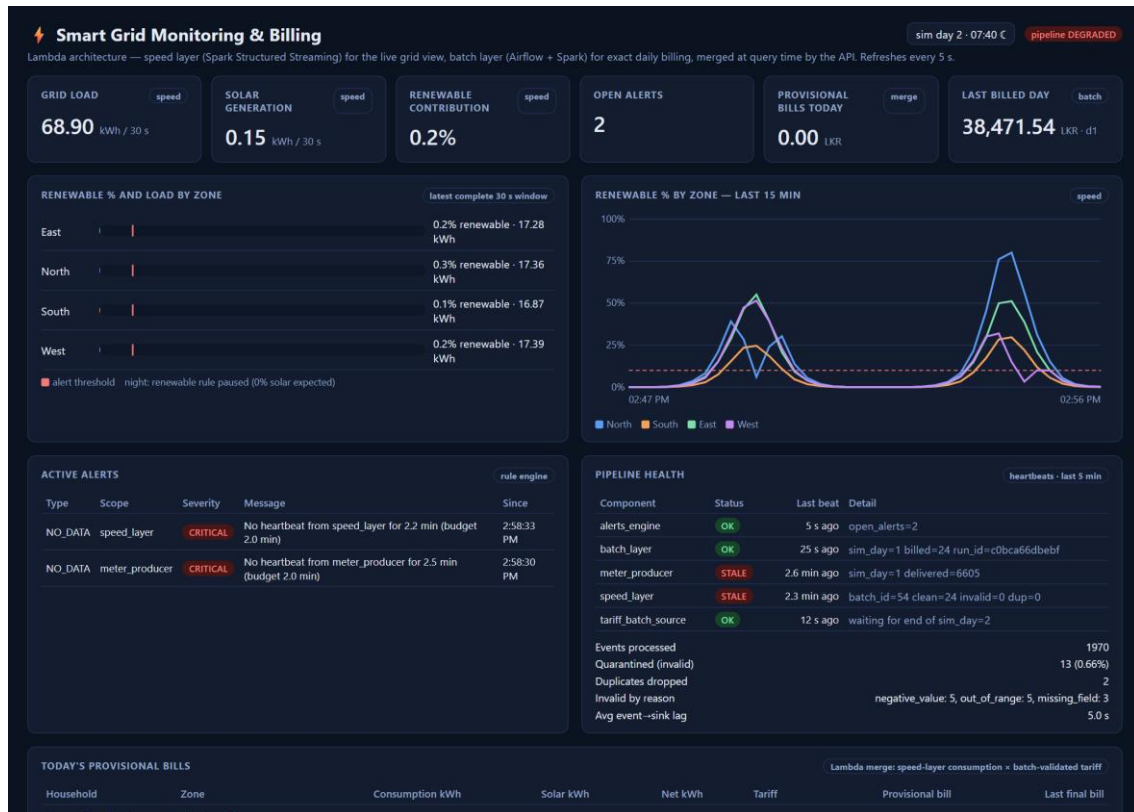


Figure 4. Dashboard during the drill: “no data” alerts and stale components.

7. Testing and Reproducibility

- **Unit tests** (no infrastructure needed) cover the data-quality rules, the billing formula, the simulated clock, both simulators (fields, value ranges, repeatability, the overcast spell, every kind of faulty reading being rejected, and the safe file drop), the alert conditions, the API endpoints with a stand-in database (the provisional bill, health staleness, the response when the database is down, and metrics) and the report generator. All 44 tests passed in about 1 second.
- **End-to-end smoke test** against the running system. It checks each layer in turn and reported the following:

Check	Result
Ingestion: producer delivered readings to Kafka	Passed: 10,490 readings delivered
Speed layer: a live window exists for every zone	Passed: all 4 zones (North, South, East, West)
Speed layer: readings processed in the last 5 minutes	Passed: 3,679 readings
Processing: faulty readings are quarantined	Passed: 16 with unreadable timestamp, 12 with impossible value, 10 with missing field, 7 with negative value
Observability: health check reports every component OK	Passed: all five components OK
Observability: metrics endpoint exports pipeline figures	Passed
Observability: Prometheus alert rules loaded	Passed: 9 rules
Observability: Prometheus can reach every metrics source	Passed: producer, API and Prometheus itself all up
Batch layer: every bill matches the billing formula	Passed: 24 bills for simulated day 4, no mismatches
Batch layer: speed-versus-batch reconciliation recorded	Passed: difference 0.14%
Lambda merge: provisional bills for the current day	Passed: 24 households, total LKR 15,491.04

- **Reproducing the results:** start the whole system with one Docker Compose command, wait about six minutes for the first bill, then run the end-to-end smoke test. All settings are controlled through environment variables.

8. Results

The results below come from one continuous run of the system (5 simulated days, about 25 minutes of real time). Every number was read from the API by an automated capture script.

8.1 Live dashboard and real-time API

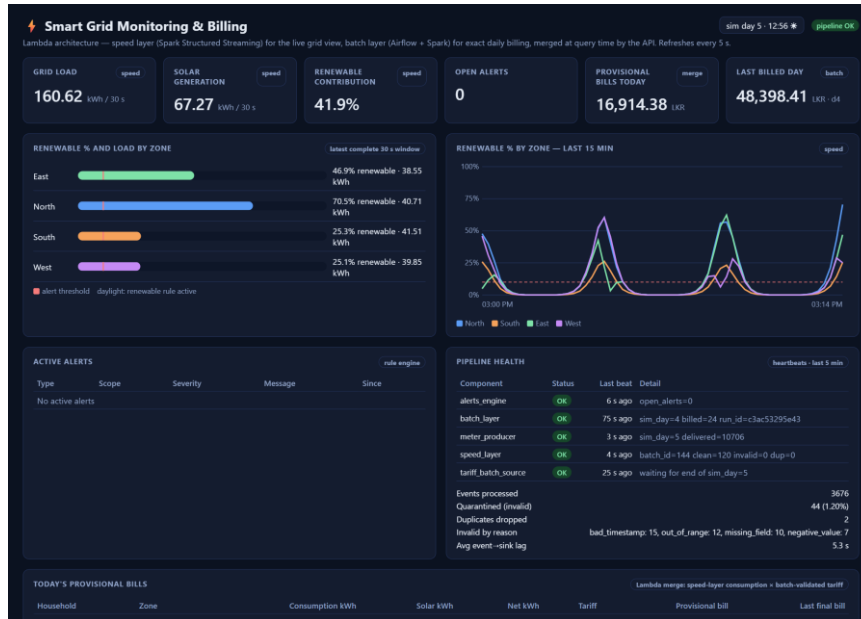


Figure 5. Live dashboard: speed-layer summary figures, renewable share per zone with the alert threshold, 15-minute history, active alerts and pipeline health.

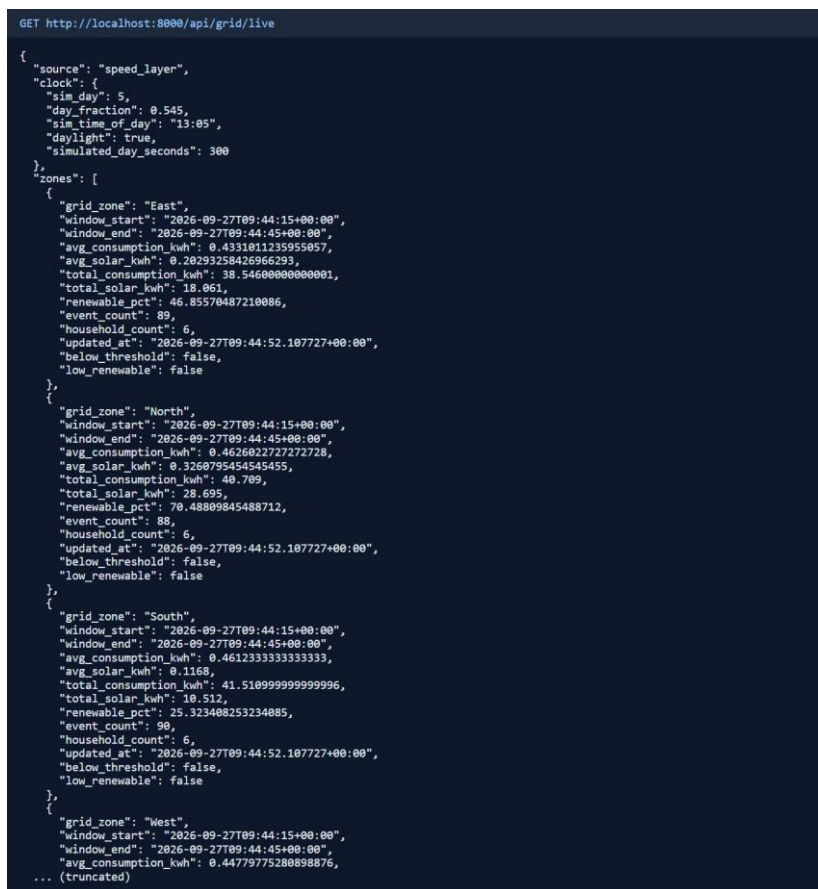


Figure 6. Live grid endpoint: the latest complete window for each zone (speed view).

8.2 Daily billing and solar-contribution report (batch view)

For simulated day 4, the batch layer billed 24 households for a total of LKR 48,398.41. Totals per zone:

Zone	Households	Consumption kWh	Solar kWh	Avg solar contribution %	Billed (LKR)
East	6	567.85	70.06	12.3	12,339.32
North	6	558.27	70.17	12.6	10,960.30
South	6	562.47	25.92	4.6	12,609.07
West	6	552.53	35.68	6.5	12,489.72

The first rows of the per-household billing report:

Household	Zone	Tier	Net kWh	Solar %	Tariff	Source	Subsidy	Bill (LKR)
H-000	North	standard	79.86	12.6	26.50	today	no	2,116.32
H-001	South	low	89.34	4.6	17.81	today	no	1,591.13
H-002	East	high	83.32	12.3	31.74	today	no	2,644.42
H-003	West	low	86.44	6.6	17.33	today	no	1,498.01
H-004	North	low	77.51	13.1	18.81	today	no	1,457.96
H-005	South	standard	87.78	4.8	26.20	today	no	2,299.94

Smart Grid - Consolidated Daily Billing & Solar Report

Simulated day 4 - generated 2026-09-27 09:43:50 UTC - batch run c3ac53295e43

Households billed 24 Total billed (LKR) 48,398.41 Consumption (kWh) 2,241.12 Solar generated (kWh) 201.83

By zone

Zone	Households	Consumption kWh	Solar kWh	Avg solar contribution %	Billed LKR
East	6	567.85	70.06	12.3	12,339.32
North	6	558.27	70.16	12.6	10,960.30
South	6	562.46	25.92	4.6	12,609.07
West	6	552.53	35.68	6.5	12,489.72

Data quality & speed/batch reconciliation

Raw events	Duplicates removed	Speed-layer kWh	Batch kWh	Drift %	Tariff rows	Invalid tariff rows	Fallback tariffs	Unbilled households
3641	4	2244.16	2241.12	<0.14	25	2	1	0

Top 5 bills

Household	Zone	Net kWh	Tariff	Bill LKR
H-007	West	91.30	31.24	2,852.24
H-011	West	85.95	31.52	2,709.14
H-002	East	83.32	31.74	2,644.42
H-018	East	84.66	31.21	2,642.27
H-017	South	91.79	26.20	2,404.79

All households

Household	Zone	Tier	Consumption	Solar	Net	Solar %	Tariff	Tariff source	Subsidy	Bill LKR
H-000	North	residential_standard	91.39	11.53	79.86	12.6	26.50	current_day	no	2,116.32
H-001	South	residential_low	93.64	4.30	89.34	4.6	17.81	current_day	no	1,591.13
H-002	East	residential_high	95.04	11.73	83.32	12.3	31.74	current_day	no	2,644.42
H-003	West	residential_low	92.52	6.08	86.44	6.6	17.33	current_day	no	1,498.01
H-004	North	residential_low	89.18	11.67	77.51	13.1	18.81	current_day	no	1,457.96
H-005	South	residential_standard	92.17	4.38	87.78	4.8	26.20	current_day	no	2,299.94
H-006	East	residential_low	91.43	11.81	79.61	12.9	17.52	current_day	no	1,394.84
H-007	West	residential_high	97.40	6.10	91.30	6.3	31.24	current_day	no	2,852.24
H-008	North	residential_low	94.51	11.43	83.08	12.1	19.18	current_day	no	1,593.46
H-009	South	residential_standard	96.53	4.34	92.19	4.5	25.29	current_day	no	2,331.46
H-010	East	residential_standard	98.40	11.76	86.64	11.9	24.88	fallback_previous_day	no	2,155.50
H-011	West	residential_high	91.69	5.74	85.95	6.3	31.52	current_day	no	2,709.14
H-012	North	residential_standard	97.56	11.97	85.60	12.3	25.39	current_day	no	2,173.33
H-013	South	residential_standard	93.27	4.32	88.95	4.6	25.40	current_day	yes	1,920.50
H-014	East	residential_standard	93.94	11.47	82.46	12.2	24.06	current_day	no	1,984.08
H-015	West	residential_low	88.10	6.00	82.10	6.8	19.43	current_day	yes	1,355.94
H-016	North	residential_standard	88.90	11.80	77.10	13.3	26.37	current_day	no	2,033.10
H-017	South	residential_standard	96.08	4.29	91.79	4.5	26.20	current_day	no	2,404.79
H-018	East	residential_high	96.53	11.86	84.66	12.3	31.21	current_day	no	2,642.27
H-019	West	residential_standard	94.92	5.88	89.04	6.2	26.45	current_day	yes	2,001.82
H-020	North	residential_low	96.72	11.76	84.96	12.2	18.67	current_day	no	1,586.13
H-021	South	residential_standard	90.78	4.28	86.50	4.7	23.83	current_day	no	2,061.25
H-022	East	residential_low	92.53	11.43	81.10	12.3	18.72	current_day	no	1,518.21
H-023	West	residential_standard	87.90	5.89	82.02	6.7	25.27	current_day	no	2,072.57

Figure 7. Daily consolidated report file, published automatically by the Airflow pipeline.

8.3 Lambda merge: provisional bills during the day

At the time of capture (simulated day 5, 11:33), household H-001 had used 28.63 kWh and generated 1.58 kWh of solar according to the speed layer. Applying its latest validated tariff from the batch layer (17.81 LKR/kWh,

from day 4) gives a provisional bill so far of LKR 481.81, shown next to its final bill of LKR 1,591.13 for day 4. The provisional total for all 24 households was LKR 15,491.04. Figure 8 shows the same endpoint for this household a few seconds later; the figures grow as new readings arrive.

```
GET http://localhost:8080/api/billing/projection?household_id=H-001

{
  "source": "lambda_merge(speed_layer consumption x batch_layer tariff)",
  "clock": {
    "sim_day": 5,
    "day_fraction": 0.546,
    "sim_time_of_day": "13:06",
    "daylight": true,
    "simulated_day_seconds": 300
  },
  "count": 1,
  "total_provisional": 560.68,
  "rows": [
    {
      "household_id": "H-001",
      "grid_zone": "South",
      "consumption_kwh": 34.403,
      "solar_kwh": 2.922,
      "reading_count": 78,
      "tariff_rate": 17.81,
      "billing_tier": "residential_low",
      "subsidy_flag": false,
      "tariff_sim_day": 4,
      "last_final_bill": 1591.13,
      "last_final_sim_day": 4,
      "net_consumption_kwh": 31.481,
      "solar_contribution_pct": 8.49,
      "provisional_bill_so_far": 560.68,
      "status": "provisional"
    }
  ]
}
```

Figure 8. Provisional bill endpoint: live consumption from the speed layer combined with the tariff from the batch layer.

8.4 Speed vs batch reconciliation and data quality

Sim day	Raw readings	Duplicates removed	Speed kWh	Batch kWh	Difference %	Invalid tariff rows	Fallback tariffs	Billed (LKR)
0	3599	0	2236.16	2236.16	0.00	0	0	48,392.51
1	3003	0	1821.50	1821.50	0.00	0	0	38,471.54
2	1716	5	1409.65	1405.80	0.27	0	0	31,798.40
3	3669	6	2271.84	2268.20	0.16	0	0	49,030.64
4	3641	4	2244.16	2241.12	0.14	2	1	48,398.41

Over the whole run the batch layer read 15,628 readings from the data lake and removed 15 duplicates (on days 2, 3, 4). These were readings delivered a second time in a later micro-batch than the original, so the speed layer, which only removes duplicates within one micro-batch, could not see them and counted them twice. A positive difference is exactly that over-count, and the batch layer's day-wide duplicate removal takes it out of the final bill. Days with no difference simply had no such duplicates. Faulty tariff files arrived on days 3 and 4: the invalid or repeated rows were rejected, and 1 household was billed with its previous valid tariff instead. Days 1 and 2 also include the three-and-a-half-minute producer outage from the failure drill (section 6.1). That lowers their consumption and bills but not their correctness, because both layers saw the same, smaller set of readings.

In the five minutes before capture, the speed layer processed 3,679 readings in 30 micro-batches. It quarantined 45 of them (1.22%: 16 with unreadable timestamp, 12 with impossible value, 10 with missing field, 7 with negative value), dropped 3 duplicates within micro-batches, and had an average delay of 5.3 seconds from a reading being taken to being stored.

8.5 Alerts, orchestration and monitoring

During the run the alert engine raised 8 “low renewable share” alerts and 2 “no data” alerts. The low-renewable alerts by zone were South 5, West 1, East 1, North 1. South, which has the fewest solar panels, drops below 10% at the start of simulated daylight every day, while the other zones only triggered during that day's overcast spell. Each alert closed itself within one or two 15-second checks once the next window recovered. The two “no data” alerts come from the failure drill in section 6.1. Because the low-renewable rule has no minimum duration, a zone hovering around the threshold can raise short-lived alerts; section 9 describes the fix.

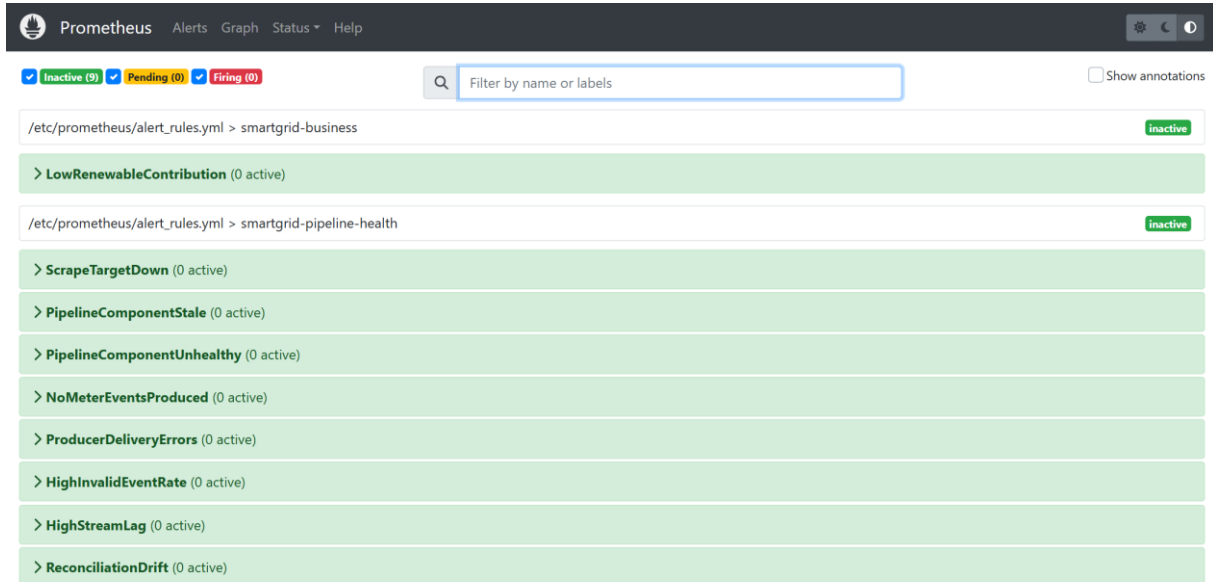


Figure 9. Prometheus: the pipeline's alert rules loaded and evaluated.

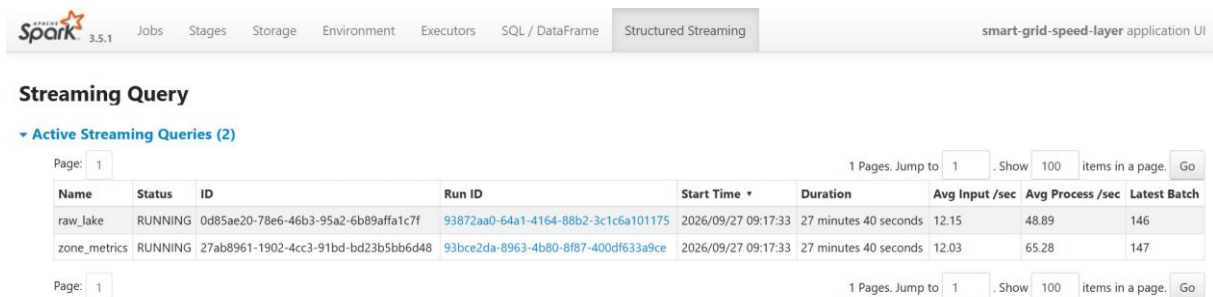


Figure 10. Spark UI: the two streaming queries of the speed layer (zone figures and raw data lake).

9. Limitations, Trade-offs and Production-Scale Changes

Area	Simplification in this project	What we would do at production scale
Kafka	One broker, a single copy of each message, no authentication	At least three brokers, three copies of each message with at least two required to confirm a write, duplicate-safe producers, encrypted and authenticated connections, and a schema registry so the message format can evolve safely
Spark	Runs on two local cores inside a container; the batch job runs inside the Airflow container	Spark on Kubernetes or YARN, launched by Airflow as a separate job, with autoscaling
Speed-layer accuracy	At-least-once: duplicates are only removed within a micro-batch, and running totals can count a reading twice after a crash	Duplicate removal across batches or transactional writes for exactly-once results, while keeping the batch layer as the source of truth
Late data	Readings more than one minute late are left out of the live windows (they are still kept in the data lake)	Set the watermark from observed delays, and re-run a day's billing if late data arrives after the bill

Area	Simplification in this project	What we would do at production scale
Airflow	Stand-alone mode, running one task at a time, with a SQLite metadata database	A distributed executor with a PostgreSQL metadata database, and runs started by the arrival of the tariff file itself
Data lake	A local Docker volume holding Parquet files	S3 or HDFS with a table format such as Delta Lake or Iceberg for safe appends, compaction and time travel
Serving	One PostgreSQL instance, no login, the dashboard refreshes every 5 seconds	Read replicas or a time-series database for live figures, API authentication, and pushed updates instead of polling
Observability	Our own alert engine and a single Prometheus, without Grafana or Alertmanager; the low-renewable rule has no minimum duration, so it can flicker near the threshold	Alert hysteresis (for example, trigger after 2 minutes below 10% and clear above 12%), Alertmanager for routing and on-call, Grafana dashboards, OpenTelemetry tracing and a central log store
Billing	A flat rate per kWh, one subsidy rule, no credit for surplus solar	Tiered and time-of-use tariffs, net-metering credits, versioned tariffs and an audit trail
Simulation	24 households and 5-minute days	Real meter volumes need more partitions and a partitioning scheme by zone and date

What we would do differently. We would introduce a schema registry from the first day, because the loosely defined JSON messages are the main source of dirty-data risk. We would use a table format such as Delta Lake so that the batch layer could replace a day directly and atomically in the data lake. And we would base the renewable alert on the solar output expected for that hour and weather, instead of a fixed 10%.

10. Conclusion

The platform takes in a continuous stream of meter readings and a daily tariff file. It answers the business question both in real time (grid load and renewable share by zone) and once a day (exact household bills and solar contribution), and it presents both views, together with their combination, through one API and one dashboard. We chose Lambda because this use case combines a need for low-latency monitoring with a daily calculation that must be correct and repeatable over a source that is naturally a batch, and the measured difference between the speed and batch results shows why the batch layer is needed. Every stage of the pipeline can be observed through structured logs, heartbeats, quality and delay metrics, seven alert rules and Prometheus.

Individual contributions

- **Sewvandi M.A.K. (EG/2021/4808):** the Lambda versus Kappa decision and the overall system design; the Spark streaming speed layer (cleaning, windowing and the raw data lake); the Spark batch layer (daily recalculation, tariff join, billing and reconciliation); Airflow orchestration; the PostgreSQL schema; and writing the report.
- **Peiris P.R.S. (EG/2021/4706):** the smart-meter and tariff data simulators; the Kafka ingestion setup; the REST API and live dashboard; observability (structured logging, alert rules and Prometheus); the Docker Compose setup; and the automated tests and results capture.